Color-to-gray Filter

Course: Massively Parallel Programming on GPUs

Teacher: Donato D'Ambrosio
Master Degree in Artificial Intelligence and Computer Science
Department of Mathematics and Computer Science
University of Calabria, Italy

October 22, 2026

1 Introduction

This CUDA lab consists of:

- Developing a baseline color-to-gray filter in OpenMP for CPU execution and a color-to-gray kernel in CUDA for GPU execution;
- Assess performance, check correctness, and analyse results.

2 Image color-to-gray graphic filter

Consider an input RGB image where each pixel consists of three unsigned 8-bit channels (r, g, b): red, green, and blue.

Implement a baseline CPU OpenMP version of the color-to-gray filter in C/C++ to be used as a reference for correctness and performance assessment. In order to obtain the gray color from the (r, g, b) values, you can use the following formula:

$$gray = 0.21 \cdot r + 0.71 \cdot g + 0.07 \cdot b$$

Implement the color-to-gray application in CUDA using standard-memory (not the unified one) and a grid-stride-loop. Use:

- the `stb_image.h` header library to load the input image;
- the `stb_image_write.h` header library to save the output image.

You can download the stb header files from the std repository.

The input image must be loaded from disk on the host. Allocate device buffers using `cudaMalloc()` and transfer the input image using `cudaMemcpy()`. The output image must be copied back to the host.

3 Performance Assessment, Correctness, and Analysis

Consider the 3072x2048 (6.3 mega-pixels, 19.44 MB) TIFF image of the Cube of Rubik, freely downloadable from the Bruce Lindbloom's website.

3.1 Performance Assessment

Assess the performance of the OpenMP implementation on one of the eight cores CPUs on the JPDM2 workstation, using all the eight cores. Measure the elapsed time of the color-to-gray computation separately from image loading and saving.

Assess the CUDA implementations on one of the NVIDIA GTX 980 GPUs installed on the JPDM2 workstation using the following block sizes:

$$B \in \{ \text{8x8}, \text{16x16}, \text{32x32} \}$$

Measure the elapsed time of the color-to-gray computation separately from image loading and saving.

Each execution must be repeated four times, and the average execution time computed and considered for performance analysis.

3.2 Correctness

Compare every CUDA output against the OpenMP reference implementation. Since both implementations perform the same operations, the output pixels should match exactly. Use the `md5sum` command to compare the checksums of CPU and GPU output files.

3.3 Performance Analysis

Produce tables and plots comparing the CPU and CUDA implementations:

- A bar plot with the CPU and GPU elapsed times by considering the three block configurations tested;
- A bar plot with the costs of memory transfers, initialization, and kernel execution for the fastest GPU execution;
- The speedup achieved by each CUDA execution with respect to the OpenMP execution.

Comment the observed results.